```
/*
================================================
==========================
  ملف واحد يجمع كل التطبيق - (Dawaei) دوائي
  في جذر المشروع. ويلزمك ملفان App.js ضعه باسم
صغيران بجانبه:

  ---------- package.json ----------
  {
    "name": "dawaei",
    "version": "1.0.0",
    "main":
"node_modules/expo/AppEntry.js",
    "dependencies": {
      "@react-native-async-storage/async-
storage": "1.23.1",
      "@react-navigation/native":
"^6.1.18",
      "@react-navigation/native-stack":
"^6.11.0",
      "expo": "~51.0.0",
      "expo-notifications": "~0.28.0",
      "expo-status-bar": "~1.12.1",
      "expo-task-manager": "~11.8.2",
      "react": "18.2.0",
      "react-native": "0.74.5",
      "react-native-immediate-phone-call":
"^1.0.0",
      "react-native-safe-area-context":
"4.10.5",
```

```
      "react-native-screens": "3.31.1"
    }
  }

  ---------- app.json ----------
  {
    "expo": {
      "name": "دوائي",
      "slug": "dawaei",
      "version": "1.0.0",
      "orientation": "portrait",
      "android": {
        "package": "com.dawaei.app",
        "permissions":
["CALL_PHONE","POST_NOTIFICATIONS","SCHEDUL
E_EXACT_ALARM","USE_FULL_SCREEN_INTENT","VI
BRATE"]
      },
      "plugins": [["expo-notifications", {
"color": "#2E7D5B" }]]
    }
  }

  التشغيل: npm install ثم npx expo prebuild ثم
npx expo run:android
  الاتصال التلقائي يعمل على أندرويد فقط ويحتاج
Development Build.
  عدّل REPORT_URL التقرير اليومي يحتاج خادم: بالأسفل.
==========================================
==========================
*/

import React, { useEffect, useState } from
'react';
```

```
import { I18nManager, Platform, Linking,
View, Text, TextInput, TouchableOpacity,
StyleSheet, ScrollView, FlatList, Alert }
from 'react-native';
import { NavigationContainer } from
'@react-navigation/native';
import { createNativeStackNavigator } from
'@react-navigation/native-stack';
import { StatusBar } from 'expo-status-
bar';
import AsyncStorage from '@react-native-
async-storage/async-storage';
import * as Notifications from 'expo-
notifications';
import * as TaskManager from 'expo-task-
manager';
import { immediatePhoneCall } from 'react-
native-immediate-phone-call';


// ================= الثيم =================
const colors = {
  green: '#2E7D5B',
  greenDark: '#1F5C42',
  greenSoft: '#E4F1EA',
  gray: '#5B6661',
  grayLight: '#EEF0EF',
  grayLine: '#D3D8D5',
  white: '#FFFFFF',
  danger: '#B3261E',
};

// خطوط كبيرة لأن المستخدم كبير سن
const type = {
```

```
  title: { fontSize: 32, fontWeight: '800',
color: colors.greenDark },
  body: { fontSize: 20, color: colors.gray,
lineHeight: 30 },
  label: { fontSize: 18, fontWeight: '700',
color: colors.greenDark },
};

// ================= التخزين
=================
const K = { user: 'dawaei_user', settings:
'dawaei_settings', meds: 'dawaei_meds',
pending: 'dawaei_pending' };

const read = async (k, fallback) => {
  try { const v = await
AsyncStorage.getItem(k); return v ?
JSON.parse(v) : fallback; }
  catch { return fallback; }
};
const write = (k, v) =>
AsyncStorage.setItem(k, JSON.stringify(v));

const getUser = () => read(K.user, null);
const saveUser = (u) => write(K.user, u);
const logout = () =>
AsyncStorage.removeItem(K.user);

const getSettings = () =>
  read(K.settings, { emergencyNumber: '',
beneficiaryNumber: '', reportEmail: '',
waitMinutes: 10 });
const saveSettings = (s) =>
write(K.settings, s);
```

```
const getMeds = () => read(K.meds, []);
const saveMeds = (m) => write(K.meds, m);

// جرعات بانتظار رد المستفيد: { [doseId]: { name,
time, taken } }
const getPending = () => read(K.pending,
{});
const savePending = (p) => write(K.pending,
p);

// ================= التقرير اليومي
=================
// عدّل الرابط إلى الخادم حقك (انظر README و server-
example.js)
const REPORT_URL = 'https://YOUR-
SERVER.example.com/daily-report';

async function sendReport(payload) {
  const { reportEmail, beneficiaryNumber }
= await getSettings();
  const user = await getUser();
  if (!reportEmail) return;
  await fetch(REPORT_URL, {
    method: 'POST',
    headers: { 'Content-Type':
'application/json' },
    body: JSON.stringify({ to: reportEmail,
beneficiaryNumber, user: user?.name,
...payload, at: new Date().toISOString()
}),
  });
}
```

```
// ================= الاتصال بالطوارئ
=================
// يتصل بالطوارئ إذا ما تم تأكيد الجرعة
async function escalateIfNotTaken(doseId) {
  const pending = await getPending();
  const dose = pending[doseId];
  if (!dose || dose.taken) return false;

  const { emergencyNumber } = await
getSettings();
  if (!emergencyNumber) return false;

  dose.missed = true;
  pending[doseId] = dose;
  await savePending(pending);
  sendReport({ type: 'missed', dose
}).catch(() => {});

  if (Platform.OS === 'android') {
    immediatePhoneCall(emergencyNumber); //
(أندرويد فقط) اتصال مباشر
  } else {

Linking.openURL(`tel:${emergencyNumber}`);
// آيفون لا يسمح بالاتصال التلقائي
  }
  return true;
}

// ================= الإشعارات
=================
const BG_TASK = 'DAWAEI_BG_NOTIFICATION';

Notifications.setNotificationHandler({
```

```
  handleNotification: async () => ({
shouldShowAlert: true, shouldPlaySound:
true, shouldSetBadge: false }),
});

// يشتغل لما يوصل إشعار "التصعيد" والتطبيق بالخلفية
(أندرويد)
TaskManager.defineTask(BG_TASK, async ({
data }) => {
  const content =
data?.notification?.request?.content ??
data?.notification?.data;
  const d = content?.data ?? data?.data ??
{};
  if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
});
Notifications.registerTaskAsync(BG_TASK).ca
tch(() => {});

async function setupNotifications() {
  const { status } = await
Notifications.requestPermissionsAsync();
  if (Platform.OS === 'android') {
    await
Notifications.setNotificationChannelAsync('
meds', {
      name: 'مواعيد الدواء',
      importance:
Notifications.AndroidImportance.MAX,
      sound: 'default',
      vibrationPattern: [0, 500, 300, 500],
      lockscreenVisibility:
Notifications.AndroidNotificationVisibility
```

```
.PUBLIC,
    });
  }
  await
Notifications.setNotificationCategoryAsync(
'dose', [
    { identifier: 'TAKEN', buttonTitle: '✅
أخذت الدواء', options: { opensAppToForeground:
false } },
  ]);
  return status === 'granted';
}

// جدولة تذكير يومي + إشعار تصعيد بعد دقائق الانتظار
async function scheduleMed(med) {
  const { waitMinutes } = await
getSettings();
  const [h, m] =
med.time.split(':').map(Number);
  const esc = new Date(2000, 0, 1, h, m +
(waitMinutes || 10));

  const reminderId = await
Notifications.scheduleNotificationAsync({
    content: {
      title: 'وقت الدواء 💊',
      body: `${med.name} ${med.dose ? '- '
+ med.dose : ''}`,
      categoryIdentifier: 'dose',
      data: { kind: 'remind', medId:
med.id, name: med.name, time: med.time },
      sound: 'default',
      ...(Platform.OS === 'android' ? {
channelId: 'meds' } : {}),
```

```
    },
    trigger: { hour: h, minute: m, repeats:
true },
  });

  const escalateId = await
Notifications.scheduleNotificationAsync({
    content: {
      title: 'لم يتم تأكيد الدواء',
      body: 'جاري الاتصال برقم الطوارئ',
      data: { kind: 'escalate', medId:
med.id, doseId: `${med.id}` },
      ...(Platform.OS === 'android' ? {
channelId: 'meds' } : {}),
    },
    trigger: { hour: esc.getHours(),
minute: esc.getMinutes(), repeats: true },
  });
  return { reminderId, escalateId };
}

async function cancelMed(med) {
  for (const id of [med.reminderId,
med.escalateId]) if (id) await
Notifications.cancelScheduledNotificationAs
ync(id);
}

// تسجيل أن الجرعة انأخذت
async function markTaken(med) {
  const pending = await getPending();
  pending[med.id] = { name: med.name, time:
med.time, taken: true, at: Date.now() };
  await savePending(pending);
```

```
  sendReport({ type: 'taken', dose:
pending[med.id] }).catch(() => {});
}

// يفتح التطبيق من الإشعار
function listenToResponses(meds) {
  const sub =
Notifications.addNotificationResponseReceiv
edListener(async (r) => {
    const d =
r.notification.request.content.data || {};
    const med = meds.find((x) => x.id ===
d.medId);
    if (!med) return;
    if (r.actionIdentifier === 'TAKEN')
await markTaken(med);
    else if (d.kind === 'remind') {
      const pending = await getPending();
      pending[med.id] = { name: med.name,
time: med.time, taken: false };
      await savePending(pending);
    } else if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
  });
  // لما يوصل إشعار التذكير نسجّل أنه بانتظار الرد
  const recv =
Notifications.addNotificationReceivedListen
er(async (n) => {
    const d = n.request.content.data || {};
    if (d.kind === 'remind') {
      const pending = await getPending();
      pending[d.medId] = { name: d.name,
time: d.time, taken: false };
      await savePending(pending);
```

```
    } else if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
  });
  return () => { sub.remove();
recv.remove(); };
}

// ================= شاشة الترحيب
=================
const FEATURES = [
  'تذكير بموعد الدواء بصوت عالٍ',
  'إذا ما رديت على التنبيه، يتصل التطبيق تلقائياً برقم
الطوارئ',
  'تقرير يومي يوصل لبريد أهلك',
];

function WelcomeScreen({ navigation }) {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] =
useState('');

  const login = async () => {
    if (!name.trim() ||
!email.includes('@') || password.length <
6) {
      Alert.alert('بيانات ناقصة', 'اكتب الاسم وبريدًا
صحيحًا وكلمة مرور من 6 خانات على الأقل.');
      return;
    }
    // تسجيل محلي للتجربة. للإنتاج استبدله بـ Firebase
Auth.
    await saveUser({ name: name.trim(),
email: email.trim() });
```

```
    navigation.replace('Settings');
  };

  return (
    <ScrollView contentContainerStyle=
{sW.wrap}
keyboardShouldPersistTaps="handled">
      <View style={sW.hero}>
        <Text style={sW.logo}>💊</Text>
        <Text style=
{sW.appName}>دوائي</Text>
        <Text style={sW.tag}>ما تنسى دواك، وأهلك
مطمنين عليك</Text>
      </View>

      <View style={sW.card}>
        {FEATURES.map((f) => (
          <View key={f} style={sW.row}>
            <View style={sW.dot} />
            <Text style={[type.body, {
flex: 1 }]}>{f}</Text>
          </View>
        ))}
      </View>

      <Text style={sW.h}>تسجيل الدخول</Text>
      <TextInput style={sW.input}
placeholder="الاسم" value={name}
onChangeText={setName} textAlign="right" />
      <TextInput style={sW.input}
placeholder="البريد الإلكتروني" value={email}
onChangeText={setEmail}
        keyboardType="email-address"
autoCapitalize="none" textAlign="right" />
```

```
      <TextInput style={sW.input}
placeholder="كلمة المرور" value={password}
onChangeText={setPassword}
        secureTextEntry textAlign="right"
/>

      <TouchableOpacity style={sW.btn}
onPress={login} accessibilityRole="button">
        <Text style=
{sW.btnText}>دخول</Text>
      </TouchableOpacity>
    </ScrollView>
  );
}

const sW = StyleSheet.create({
  wrap: { flexGrow: 1, backgroundColor:
colors.grayLight, paddingBottom: 40 },
  hero: { backgroundColor: colors.green,
paddingTop: 70, paddingBottom: 40,
alignItems: 'center',
borderBottomLeftRadius: 36,
borderBottomRightRadius: 36 },
  logo: { fontSize: 64 },
  appName: { fontSize: 44, fontWeight:
'900', color: colors.white, marginTop: 4 },
  tag: { fontSize: 19, color:
colors.greenSoft, marginTop: 6 },
  card: { backgroundColor: colors.white,
margin: 20, padding: 18, borderRadius: 18,
gap: 12 },
  row: { flexDirection: 'row', alignItems:
'flex-start', gap: 10 },
  dot: { width: 12, height: 12,
```

```
borderRadius: 6, backgroundColor:
colors.green, marginTop: 9 },
  h: { ...type.title, fontSize: 26,
marginHorizontal: 20, marginBottom: 8 },
  input: { backgroundColor: colors.white,
borderWidth: 1.5, borderColor:
colors.grayLine, borderRadius: 14,
marginHorizontal: 20, marginBottom: 12,
paddingHorizontal: 16, paddingVertical: 14,
fontSize: 20 },
  btn: { backgroundColor: colors.green,
marginHorizontal: 20, marginTop: 6,
padding: 18, borderRadius: 14, alignItems:
'center' },
  btnText: { color: colors.white, fontSize:
22, fontWeight: '800' },
});

// ================= شاشة بيانات الطوارئ
=================
const isPhone = (v) => /^\+?[0-9]
{8,15}$/.test(v.replace(/\s/g, ''));

function SettingsScreen({ navigation }) {
  const [emergencyNumber, setEmergency] =
useState('');
  const [beneficiaryNumber, setBeneficiary]
= useState('');
  const [reportEmail, setEmail] =
useState('');

  useEffect(() => {
    getSettings().then((s) => {
      setEmergency(sS.emergencyNumber);
```

```
      setBeneficiary(sS.beneficiaryNumber);
      setEmail(sS.reportEmail);
    });
  }, []);

  const save = async () => {
    if (!isPhone(emergencyNumber) ||
!isPhone(beneficiaryNumber) ||
!reportEmail.includes('@')) {
      Alert.alert('تأكد من البيانات', 'اكتب رقمين
صحيحين (9665+ مثال: XXXXXXXX) وبريدًا إلكترونيًا
صحيحًا.');
      return;
    }
    const old = await getSettings();
    await saveSettings({ ...old,
emergencyNumber: emergencyNumber.trim(),
beneficiaryNumber:
beneficiaryNumber.trim(), reportEmail:
reportEmail.trim() });
    navigation.navigate('Meds');
  };

  const out = async () => { await logout();
navigation.replace('Welcome'); };

  return (
    <ScrollView contentContainerStyle=
{sS.wrap}
keyboardShouldPersistTaps="handled">
      <Text style={sS.label}>رقم
الطوارئ</Text>
      <Text style={sS.hint}>يتصل عليه التطبيق إذا
ما رديت على تنبيه الدواء</Text>
```

```
      <TextInput style={sS.input} value=
{emergencyNumber} onChangeText=
{setEmergency} keyboardType="phone-pad"
placeholder="+9665XXXXXXXX"
textAlign="right" />

      <Text style={sS.label}>رقم
المستفيد</Text>
      <Text style={sS.hint}>رقم جوال كبير السن
الذي يستخدم الدواء</Text>
      <TextInput style={sS.input} value=
{beneficiaryNumber} onChangeText=
{setBeneficiary} keyboardType="phone-pad"
placeholder="+9665XXXXXXXX"
textAlign="right" />

      <Text style={sS.label}>البريد الإلكتروني
للتقرير اليومي</Text>
      <Text style={sS.hint}>يوصله كل يوم ملخص
الجرعات المأخوذة والفائتة</Text>
      <TextInput style={sS.input} value=
{reportEmail} onChangeText={setEmail}
keyboardType="email-address"
autoCapitalize="none"
placeholder="name@example.com"
textAlign="right" />

      <TouchableOpacity style={sS.btn}
onPress={save}><Text style={sS.btnText}>حفظ
ومتابعة</Text></TouchableOpacity>
      <TouchableOpacity style={sS.ghost}
onPress={out}><Text style=
{sS.ghostText}>تسجيل الخروج</Text>
</TouchableOpacity>
```

```
    </ScrollView>
  );
}

const sS = StyleSheet.create({
  wrap: { padding: 20, backgroundColor:
colors.grayLight, flexGrow: 1 },
  label: { ...type.label, marginTop: 16 },
  hint: { fontSize: 16, color: colors.gray,
marginBottom: 8, marginTop: 2 },
  input: { backgroundColor: colors.white,
borderWidth: 1.5, borderColor:
colors.grayLine, borderRadius: 14,
paddingHorizontal: 16, paddingVertical: 14,
fontSize: 20 },
  btn: { backgroundColor: colors.green,
padding: 18, borderRadius: 14, alignItems:
'center', marginTop: 28 },
  btnText: { color: colors.white, fontSize:
22, fontWeight: '800' },
  ghost: { padding: 16, alignItems:
'center', marginTop: 8 },
  ghostText: { color: colors.gray,
fontSize: 18 },
});

// ================= شاشة الأدوية
=================
function MedsScreen() {
  const [meds, setMeds] = useState([]);
  const [pending, setPending] =
useState({});
  const [name, setName] = useState('');
  const [dose, setDose] = useState('');
```

```
  const [time, setTime] = useState('');

  const refresh = async () => {
setMeds(await getMeds()); setPending(await
getPending()); };
  useEffect(() => { refresh(); const t =
setInterval(refresh, 5000); return () =>
clearInterval(t); }, []);
  useEffect(() => listenToResponses(meds),
[meds]);

  const add = async () => {
    if (!name.trim() || !/^([01]?\d|2[0-
3]):[0-5]\d$/.test(time)) {
      Alert.alert('بيانات ناقصة', 'اكتب اسم الدواء
ووقتًا بصيغة 24 ساعة مثل 08:30 أو 21:00');
      return;
    }
    const med = { id: String(Date.now()),
name: name.trim(), dose: dose.trim(), time
};
    Object.assign(med, await
scheduleMed(med));
    const next = [...meds, med].sort((a, b)
=> a.time.localeCompare(b.time));
    await saveMeds(next); setMeds(next);
setName(''); setDose(''); setTime('');
  };

  const remove = async (m) => {
    await cancelMed(m);
    const next = meds.filter((x) => x.id
!== m.id);
    await saveMeds(next); setMeds(next);
```

```
  };

  const taken = async (m) => { await
markTaken(m); refresh(); };

  return (
    <View style={sM.wrap}>
      <View style={sM.form}>
        <TextInput style={sM.input}
placeholder="اسم الدواء" value={name}
onChangeText={setName} textAlign="right" />
        <View style={sM.line}>
          <TextInput style={[sM.input, {
flex: 1 }]} placeholder="الجرعة (حبة)" value=
{dose} onChangeText={setDose}
textAlign="right" />
          <TextInput style={[sM.input, {
flex: 1 }]} placeholder="الوقت 08:30" value=
{time} onChangeText={setTime}
keyboardType="numbers-and-punctuation"
textAlign="right" />
        </View>
        <TouchableOpacity style={sM.btn}
onPress={add}><Text style={sM.btnText}>إضافة
دواء</Text></TouchableOpacity>
      </View>

      <FlatList
        data={meds}
        keyExtractor={(m) => m.id}
        contentContainerStyle={{ padding:
16, gap: 12 }}
        ListEmptyComponent={<Text style=
{[type.body, { textAlign: 'center',
```

```
marginTop: 30 }]}>أضف أول دواء ليصلك
التذكير</Text>}
        renderItem={({ item }) => {
          const p = pending[item.id];
          return (
            <View style={sM.card}>
              <View style={{ flex: 1 }}>
                <Text style={sM.medName}>
{item.name}</Text>
                <Text style={type.body}>
{item.time} {item.dose ? `• ${item.dose}` :
''}</Text>
                {p?.missed && <Text style=
{sM.missed}>لم يتم تأكيدها</Text>}
                {p?.taken && <Text style=
{sM.ok}>تم أخذ الدواء ✓</Text>}
              </View>
              <View style={{ gap: 8 }}>
                <TouchableOpacity style=
{sM.take} onPress={() => taken(item)}><Text
style={sM.takeText}>أخذته</Text>
</TouchableOpacity>
                <TouchableOpacity onPress=
{() => remove(item)}><Text style=
{sM.del}>حذف</Text></TouchableOpacity>
              </View>
            </View>
          );
        }}
      />
    </View>
  );
}
```

```
const sM = StyleSheet.create({
  wrap: { flex: 1, backgroundColor:
colors.grayLight },
  form: { backgroundColor: colors.white,
padding: 16, gap: 10, borderBottomWidth: 1,
borderColor: colors.grayLine },
  line: { flexDirection: 'row', gap: 10 },
  input: { backgroundColor:
colors.grayLight, borderRadius: 12,
paddingHorizontal: 14, paddingVertical: 12,
fontSize: 19 },
  btn: { backgroundColor: colors.green,
padding: 14, borderRadius: 12, alignItems:
'center' },
  btnText: { color: colors.white, fontSize:
20, fontWeight: '800' },
  card: { backgroundColor: colors.white,
borderRadius: 16, padding: 16,
flexDirection: 'row', alignItems: 'center',
borderStartWidth: 6, borderStartColor:
colors.green },
  medName: { fontSize: 24, fontWeight:
'800', color: colors.greenDark },
  take: { backgroundColor:
colors.greenSoft, paddingVertical: 10,
paddingHorizontal: 18, borderRadius: 10 },
  takeText: { color: colors.greenDark,
fontSize: 18, fontWeight: '800', textAlign:
'center' },
  del: { color: colors.gray, fontSize: 16,
textAlign: 'center' },
  missed: { color: colors.danger, fontSize:
17, fontWeight: '700', marginTop: 4 },
  ok: { color: colors.green, fontSize: 17,
```

```
fontWeight: '700', marginTop: 4 },
});

// ================= التطبيق
=================
// واجهة عربية من اليمين لليسار (تحتاج إعادة تشغيل التطبيق
أول مرة)
I18nManager.allowRTL(true);
I18nManager.forceRTL(true);

const Stack = createNativeStackNavigator();

export default function App() {
  const [ready, setReady] =
useState(false);
  const [loggedIn, setLoggedIn] =
useState(false);

  useEffect(() => {
    (async () => {
      setLoggedIn(!!(await getUser()));
      await setupNotifications();
      setReady(true);
    })();
  }, []);

  if (!ready) return null;

  return (
    <NavigationContainer>
      <StatusBar style="light"
backgroundColor={colors.greenDark} />
      <Stack.Navigator
        initialRouteName={loggedIn ?
```

```
'Settings' : 'Welcome'}
        screenOptions={{
          headerStyle: { backgroundColor:
colors.greenDark },
          headerTintColor: colors.white,
          headerTitleStyle: { fontSize: 22,
fontWeight: '800' },
          headerTitleAlign: 'center',
        }}
      >
        <Stack.Screen name="Welcome"
component={WelcomeScreen} options={{
headerShown: false }} />
        <Stack.Screen name="Settings"
component={SettingsScreen} options={{
title: 'بيانات الطوارئ' }} />
        <Stack.Screen name="Meds"
component={MedsScreen} options={{ title:
'أدويتي' }} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```